The Unwritten Contract of Cloud-based Elastic Solid-State Drives

Yingjia Wang and Ming-Chang Yang
The Chinese University of Hong Kong

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132948

***Abstract*—Elastic block storage (EBS) with the storage-compute disaggregated architecture stands as a pivotal piece in today's cloud. EBS furnishes users with storage capabilities through the elastic solid-state drive (ESSD). Nevertheless, despite the widespread integration into cloud services, the absence of a thorough ESSD performance characterization raises critical doubt: when more and more services are shifted onto the cloud, can ESSD satisfactorily substitute the storage responsibilities of the local SSD and offer comparable performance?**

In this paper, we for the first time target this question by characterizing two ESSDs from Amazon AWS and Alibaba Cloud. We present an unwritten contract of cloud-based ESSDs, encapsulating four observations and five implications for cloud storage users. Specifically, the observations are counter-intuitive and contrary to the conventional perceptions of what one would expect from the local SSD. The implications we hope could guide users in revisiting the designs of their deployed cloud software, i.e., harnessing the distinct characteristics of ESSDs for better system performance.

I. Introduction

Elastic block storage (EBS) has become the cornerstone of today's cloud, underpinning the high-performance storage demand from diverse cloud services [1]–[5]. As shown in Figure 1, EBS typically adopts a storage-compute disaggregated architecture, with compute and storage clusters located separately and interconnected by the high-speed datacenter network. In EBS, the virtual machines (VM) for users are hosted in the compute cluster while the user data is persistently stored in the storage cluster. This architectural innovation not only enhances the efficiency and elasticity of the cloud infrastructure, but also improves the cost-effectiveness for users on a flexible and pay-as-you-go basis.

EBS provides storage resources for users in the form of *elastic solid-state drive*, which, for simplicity, is called *ESSD* in the following. ESSD is a virtualized storage device attached to the user VM, employing the block interface and supporting random access (e.g., read and write) in the storage space. From the perspective of the user, ESSD is similar to the well-known local SSD, on which the existing filesystems and applications can be directly deployed. But differently, the most noteworthy feature of ESSD is that it breaks the physical performance and capacity boundaries of the local SSD. This makes it possible to elastically guarantee performance level (e.g., in throughput and IOPS) and allocate storage capacity, to accommodate the ever-changing demands of users.

This work is supported in part by The Research Grants Council of Hong Kong SAR (Project No. CUHK14218522).

The rapid development of cloud computing has fostered the widespread adoption of ESSDs in numerous cloud services. *However, to the best of our knowledge, there has been no comprehensive performance characterization of cloud-based ESSDs.* As the majority of host software now is designed and optimized for the local SSD, this research gap raises concerns in the following question: *when more and more services are shifted onto the cloud, can ESSD satisfactorily substitute the storage responsibilities of the local SSD and offer comparable performance?*

To this end, we make the first endeavor to answer this question by characterizing two ESSDs from two leading cloud storage providers in the world: Amazon AWS [6] and Alibaba Cloud [7]. Based on extensive experiments, we surprisingly find that ESSD exhibits several counter-intuitive performance features, contrary to the common beliefs of the local SSD.

The Unwritten Contract of Cloud-based ESSDs

Observations

1) *The latency of ESSDs is tens to a hundred times higher than that of SSD when I/Os are not well scaled up (i.e., I/O size is small and/or I/O queue depth is low).*
2) *The performance impact of GC appears much later or even disappears.*
3) *The throughput of random writes outperforms that of sequential writes, reaching a maximum of 1.52× and 2.79× in two ESSDs, respectively.*
4) *The maximum bandwidth is deterministic and no longer sensitive to the access pattern.*

Implications

1) *Scale the I/O sizes and I/O queue depths up as much as possible.*
2) *Reconsider if and how existing GC-mitigated techniques based on local SSDs should be correspondingly adapted for ESSDs.*
3) *Rethink if it is still worthwhile to convert random writes in random-write-based software into sequential writes and if it is beneficial to proactively trigger random writes in sequential-write-based software.*
4) *Smooth the read/write I/Os to be evenly distributed across the timeline and below the guaranteed throughput budget.*
5) *Re-evaluate I/O reduction techniques (e.g., compression, deduplication) that were previously considered to impair performance.*

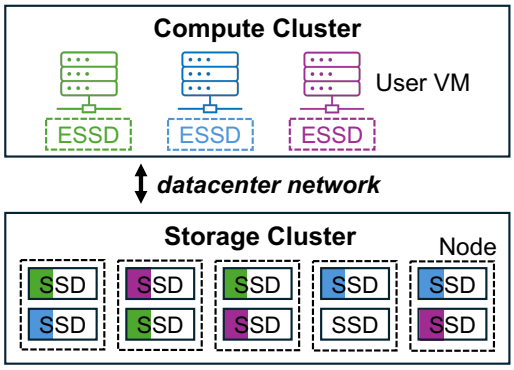


Fig. 1. **The storage-compute disaggregated architecture of elastic block storage (EBS).** ESSD is attached to the user virtual machine (VM) in the compute cluster. The physical storage space of an ESSD is distributed and replicated across different nodes and SSDs in the storage cluster.

We present an unwritten contract of cloud-based ESSDs, organized into four observations and five implications, as shown above. Over the past years, cloud storage users have been requesting ever-higher service-level agreements (SLA). This places higher expectations not only on cloud storage providers but also on users themselves in terms of their deployed software. The unwritten contract we offer in this paper could benefit cloud storage users to revisit their cloud software designs, specifically, to unleash the better potential of ESSDs by leveraging unique performance characteristics. We open-source the evaluation framework to stimulate the following studies[1].

In the following of this paper, we first provide the background of (local) SSD, EBS, and ESSD in §II. We present the unwritten contract in detail in §III. The related work and conclusion are introduced in §IV and §V, respectively.

II. Background

A. Flash-based Solid-State Drive (SSD)

Flash-based SSD has achieved a high and continuously increasing share in the storage market due to the benefits in performance, energy efficiency, and impact resistance. According to IDC, the byte shipment share of SSDs is expected to reach about 40% of the total in 2025 [8]. Flash-based SSD typically adopts a multi-level architecture, including channel, die, plane, block, and page. The flash die is the minimum unit of parallel operations while the flash page is the minimum unit of data storage. To improve SSD firmware management efficiency, flash blocks are typically grouped into superblocks, each of which comprises flash blocks across many flash dies to fully leverage flash parallelism.

To communicate with the host, the SSD longstandingly employs the *block interface*, which abstracts the storage space into an array of logical blocks (e.g., 4KB) that can be randomly read or written. While this simple abstraction has successfully contributed to the broad adoption of SSDs, a complex *flash translation layer (FTL)* is required in the SSD to bridge the gap with the underlying flash characteristics (e.g., erase-before-write). The two main duties of FTL are address mapping and garbage collection (GC). The address mapping converts logical addresses from the host to the physical addresses in the SSD by keeping track of a fine-grained (e.g., page-level) mapping table. The GC is carried out periodically to reclaim invalid space in the granularity of flash blocks, when the valid pages in some blocks are relocated and these blocks can be erased. FTL is also responsible for other tasks such as wear-leveling [9], error correction [10], and bad block management [11].

B. Elastic Block Storage (EBS)

As shown in Figure 1, *elastic block storage (EBS)* typically employs a storage-compute disaggregated architecture with independent compute and storage clusters, and the two clusters are not physically co-located but interconnected with the high-speed datacenter network. The *compute cluster* hosts virtual machines (VM) for users and forwards user requests to the backend storage cluster, while the *storage cluster* persists or fetches requested data in a distributed filesystem.

The disaggregation architecture of storage and compute brings out several benefits [1], [3], [5]. First, the storage and compute resources can be elastically paid and allocated according to the user target workloads. This allows for higher resource utilization while enhancing the cost-effectiveness for users, compared to offering general-purpose servers. Second, the resources in storage and compute clusters can be maintained and scaled independently, thereby realizing higher maintainability and scalability. Third, it is simpler and faster to migrate application services across compute servers, given that their states can be persistently stored in storage servers.

C. Elastic Solid-State Drive (ESSD)

EBS provides storage resources for users in the form of *elastic solid-state drive (ESSD)*, which is a virtualized storage device attached to user VM. ESSD can be created, attached, unattached, and destroyed without disrupting the user VM. Figure 1 illustrates the mapping relationship between the virtualized ESSD and the (multiple) physical SSDs. That is, the physical storage space of an ESSD is typically distributed and replicated (e.g., three-way [3]) across different nodes and SSDs in the storage cluster, for the purposes of load balancing and data availability.

From the user's point of view, ESSD is similar to the well-known local SSD in that ESSD also employs the block interface and supports random access in the storage space. This ensures seamless compatibility with the existing software ecosystem, and facilitates the existing file systems and applications can be directly deployed on ESSDs.

Nonetheless, beyond the local SSD, ESSD delivers several additional benefits [3], [6], [7]. First, ESSD is virtualized, which breaks the physical performance and capacity boundaries in the local SSD. As a result, both the performance level (e.g., in throughput and IOPS) and the storage capacity can

[1]https://github.com/yingjia-wang/UC

TABLE I
THE CONFIGURATIONS OF TWO ESSDS AND SSD.

	Provider and Type	Max. BW (GB/s)	Max. IOPS	Cap. (TB)	VM Type	Region
ESSD-1	Amazon AWS io2 [12]	∼3.0	25.6K	2	m6in.xlarge	Tokyo
ESSD-2	Alibaba Cloud PL3 [7]	∼1.1	100K	2	ecs.g5.4xlarge	Hangzhou
SSD	Samsung 970 Pro [13]	Seq. R/W 3.5/2.7	Rand. R/W 500K/500K (4KB, QD32)	1	/	/

be flexibly requested in response to the ever-changing user requirements. Second, ESSD can ensure high data availability and reliability, thanks to the data replication across multiple nodes to prevent single-point failure. Third, ESSD also enables some advanced features such as encryption and snapshotting.

Cloud storage providers typically offer multiple ESSD types with varying performance levels. Different performance levels guarantee different budgets in maximum throughput and maximum IOPS (higher is usually more expensive). For reference, Amazon AWS [6] supports general-purposes types (i.e., gp2, gp3) and provisioned-IOPS types (i.e., io1, io2), where io2 is the most advanced one. Alibaba Cloud [7] supports four performance levels: PL0, PL1, PL2, and PL3 (the higher the number the better the performance).

III. UNWRITTEN CONTRACT OF CLOUD-BASED ESSDS

Despite the prevalent integration in diverse cloud services, to the best of our knowledge, the performance features of the ESSD have not been adequately investigated and revealed. In this section, we present an unwritten contract of cloud-based ESSDs. In the following, we begin with the experimental setups, and then introduce our unwritten contract in detail.

Note that this paper focuses on the device-level performance characterization as in prior work [14]–[16] without exploring specific pieces of software, in the hope of benefiting a wider range of cloud storage users to revisit their software designs.

A. Experimental Setups

To strengthen the generalizability of findings, we characterize two ESSDs from two leading cloud providers in the world (i.e., Amazon AWS [6] and Alibaba Cloud [7]), and the configurations of ESSDs are summarized in Table I. Specifically, both ESSDs have the highest-class types available from their providers, making their maximum bandwidth comparable to that of the local SSD.

To demonstrate the features of the local SSD for comparison, we also evaluate Samsung 970 Pro [13], a popular SSD in research, and its configurations are also listed in Table I. Experiments on Samsung 970 Pro are running on a workstation PC, equipped with Intel® Xeon® W-2245 Processor, where the operating system is 64-bit Ubuntu 20.04 LTS with Linux kernel version 5.15.0.

We employ the widely-used FIO benchmark tool [17] to generate workloads with diverse loads (i.e., I/O size and I/O queue depth) and access patterns (i.e., random write, sequential write, random read, and sequential read). The detailed workload settings are introduced in each subsection below.

B. Latency Performance

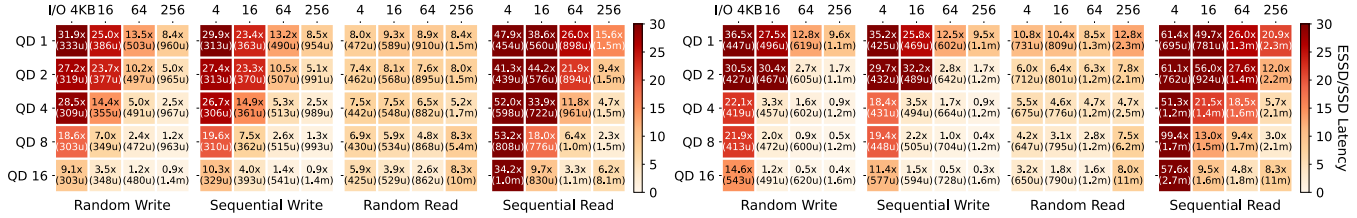
Observation #1: *The latency of ESSDs is tens to a hundred times higher than that of SSD when I/Os are not well scaled up (i.e., I/O size is small and/or I/O queue depth is low).*

ESSD can meet the user's best expectations if it can provide comparable performance (e.g., throughput and latency) to the local SSD. However, even though the maximum throughput that ESSD can provide reaches the level of GBs, we discover that the latency of ESSDs is extremely unsatisfactory when I/Os are not well scaled up. Here, we define *"not well scaled up"* as workloads dominated by small-size I/Os and/or low I/O queue depths. Figure 2 demonstrates the average and P99.9 latency of two ESSDs under different access patterns (i.e., random write, sequential write, random read, and sequential read) with I/O sizes varied from 4KB to 256KB and I/O queue depths varied from 1 to 16. Particularly, let us focus on the multiples the ESSD latency is divided by the SSD latency (i.e., the top of each pixel in Figure 2), with smaller numbers being better. For simplicity, we call this metric *latency gap* in the following of this subsection.

We can first observe that, the latency gap effectively decreases when I/Os are scaled in either sizes or queue depths. This finding is generally consistent among two metrics (i.e., average and P99.9 latency), four access patterns (i.e., random write, sequential write, random read, and sequential read), and two ESSD providers (i.e., Amazon AWS and Alibaba Cloud). Specifically, the average latency gaps of two ESSDs are up to 47.9× and 16.7×, respectively, while the P99.9 latency gaps are even much higher at up to 99.4× and 104.2×. In comparison, when the I/O size is scaled up to 256KB, the average latency gaps of two ESSDs are reduced to up to 15.6× and 7.6×, respectively, while the P99.9 latency gaps are mitigated to 20.9× and 35.1× at most. This trend also holds true when scaling up the I/O queue depths.

The primary cause of the high latency of ESSDs would be the network latency and software processing overhead within the cloud storage, which is still substantial compared to the local storage. Scaling the I/O sizes and I/O queue depths can effectively reduce the latency gap by leveraging parallelism in I/O processing and storage access distribution (see §II-C).

We can also notice that, the latency gaps in two ESSDs are significantly smaller in the random read workload but much higher in the other three workloads (i.e., random write, sequential write, and sequential read). Taking ESSD-1 as an example, the average and P99.9 latency gaps are up to 9.3× and 12.8× in the random read workload, but as high as 31.9×



(a) Average Latency of ESSD-1 from Amazon AWS

(b) P99.9 Latency of ESSD-1 from Amazon AWS

(c) Average Latency of ESSD-2 from Alibaba Cloud

(d) P99.9 Latency of ESSD-2 from Alibaba Cloud

Fig. 2. **Latency performance of two ESSDs under workloads with different access patterns (i.e., random write, sequential write, random read, sequential read), I/O sizes, and I/O queue depths (QD).** The top of each pixel indicates the multiples the ESSD latency is divided by the SSD latency, with smaller numbers being better. The bottom of each pixel indicates the value of average or P99.9 latency of the ESSD (u: μs, m: ms).

and 36.5×, 29.9× and 35.2×, 53.2× and 99.4× in the other three workloads.

This performance discrepancy is because modern SSD typically employs a DRAM-based write buffer to improve (both random/sequential) write performance and also uses prefetching to enhance sequential read performance [18]. Thus, in random/sequential write and sequential read workloads, there are much fewer flash accesses on the critical I/O path. However, the prefetching techniques work poorly for random reads, resulting in a larger number of flash accesses. This would make the inherent overhead in cloud storage (e.g., network latency, software processing) less prominent than in the other three workloads, thereby leading to a smaller latency gap.

The above results and analysis suggest that, to save the cloud software from tens to a hundred times latency penalty, cloud storage users should revisit their software designs to *scale the sizes and queue depths of I/Os up as much as possible* (**Implication #1**). Especially, when I/Os are well scaled up in random and sequential write workloads, ESSD-1 even achieves up to 3.3× better P99.9 latency compared to the local SSD (see Figure 2b).

C. Garbage Collection (GC)

> ***Observation #2:*** *The performance impact of GC appears much later or even disappears.*

Device-side garbage collection (GC) is widely recognized as one of the main culprits for SSD performance degradation and unpredictability [19]–[21]. The main reason behind this is that, to reclaim the space occupied by the invalid data, FTL periodically relocates the valid data in some flash blocks to others. The relocation of data results in a large number of extra writes, which severely compete with foreground I/Os and thus degrade the user-perceived performance. Nevertheless, in contrast to this common anticipation, we find that the performance impact of GC appears much later or even disappears in ESSDs. Figure 3 depicts the runtime throughput of two ESSDs and SSD under a random write workload, and the total write volume is configured to be 3× storage capacity of each device.

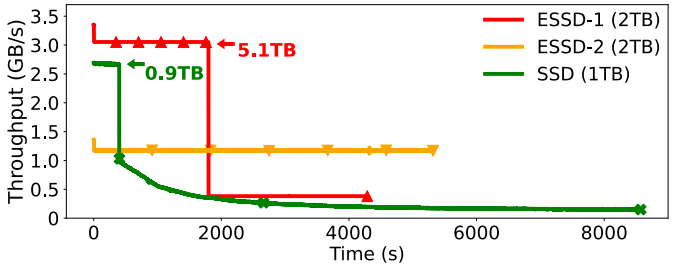


Fig. 3. **Runtime throughput of two ESSDs and SSD under a random write workload until writing 3× storage capacity.** Thus, the total write volumes towards ESSDs and SSD are 6TB and 3TB, respectively. The markers indicate the time when the total write volume reaches a multiple of 1TB.

We can first verify that, the SSD starts to experience a severe throughput drop when only writing 0.9TB (i.e., 90% storage capacity), and the throughput sharply decreases by 63% from 2.7GB/s to 1.0GB/s. After that, the throughput of SSD continues to decrease further as the write volume increases, down to a low of 149MB/s finally. The long-term low performance here demonstrates the sensitivity and powerlessness of the SSD performance to GC. Due to the high and constant write load, GC is unavoidably executed in the SSD almost all the time.

In contrast, ESSD can sustain high throughput over a much longer period even in this high-pressure workload, although the exact performance of an ESSD may depend on the provider. Specifically, ESSD-1 undergoes a sharp throughput

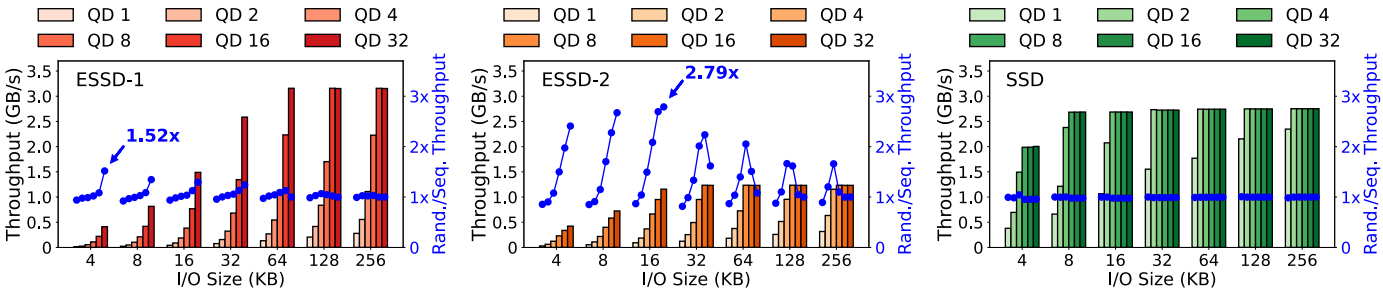


Fig. 4. **Throughput of ESSDs and SSD under random write workloads and throughput gain of ESSDs and SSD in random writes over sequential writes.** The I/O size varies from 4KB to 256KB and the I/O queue depth (QD) varies from 1 to 32.

decline when writing as large as 5.1TB (i.e., 2.55× storage capacity), after which the throughput is stabilized at about 305MB/s. On the contrary, ESSD-2 can maintain a high throughput consistently until writing 6TB (i.e., 3× storage capacity). We speculate on the rationale behind this as follows. On the one hand, cloud providers make efforts to utilize their plenty of storage resources to hide the GC impact transparently for better user experiences. On the other hand, cloud providers may trigger flow-limiting mechanisms when they can not hide the GC impact anymore.

Actually, it has been a long journey for researchers to study software techniques that can mitigate the performance impact of GC [22]–[24]. However, our results and analysis indicate that, cloud software should *reconsider if and how existing GC-mitigated techniques based on local SSDs should be correspondingly adapted for ESSDs* (**Implication #2**), due to the trade-offs they commonly entail.

D. Access Pattern

> ***Observation #3:*** *The throughput of random writes actually outperforms that of sequential writes, reaching a maximum of 1.52× and 2.79× in two ESSDs, respectively.*

Random writes have long been considered harmful to the SSD due to the exacerbated mix of valid/invalid data in the same flash blocks, which consequently results in higher valid data relocation overhead [25]–[27]. However, surprisingly, we discover that the throughput of the ESSD is generally better in the case of random rather than sequential writes. As shown in Figure 4, let us focus on the throughput gain of ESSDs and SSD in random writes over sequential writes (i.e., blue lines). For simplicity, we call this metric *throughput gain* in the following of this subsection. The I/O size varies from 4KB to 256KB and the I/O queue depth varies from 1 to 32.

We can first observe that, the throughput gains in two ESSDs are up to 1.52× and 2.79×, respectively. On the contrary, the throughput of the SSD does not show obvious differences in random or sequential writes (when GC does not occur). The main reason behind this would be the unique physical storage space mapping of ESSDs. As introduced in §II-C, the storage space of an ESSD is distributed and replicated across different nodes and SSDs in the storage cluster. Therefore, data from random writes is likely to embrace higher available SSD bandwidth, potentially leading to higher user-perceived throughput.

Additionally, we discover a notable variation in throughput gains amongst ESSD providers. Specifically, ESSD-1 has a smaller throughput gain, which is mainly concentrated on workloads with higher I/O queue depths and small-to-medium I/O sizes. For example, when the I/O size ranges from 4KB to 32KB and the I/O queue depth is 32, the throughput gain of ESSD-1 is 24.1%∼51.8%. In contrast, the throughput gain in ESSD-2 is much more significant, at up to 1.66× to 2.79× across a wide range of I/O sizes (i.e., 4KB to 256KB). The throughput gain in ESSD-2 also varies a lot in different workload scenarios. For small-size I/Os (e.g., 4KB to 16KB), the throughput gain generally increases as the queue depth increases. For medium-to-large I/Os (e.g., 32KB and larger), the throughput gain has a trend of growing and then declining as the queue depth increases; moreover, the peak of the throughput gain occurs earlier as the I/O size increases.

Converting random writes into sequential writes (e.g., log-structured [28] or copy-on-write [29]) has been widely recognized to reduce device-side GC overhead and improve SSD performance. However, given the ESSD performance is quite insensitive to GC (see §III-C) and the performance gain from random writes, it is now essential to *rethink if it is worthwhile to convert random writes in random-write-based software into sequential writes anymore and if it is beneficial to proactively trigger random writes in sequential-write-based software* (**Implication #3**).

E. Throughput Budget

> ***Observation #4:*** *The maximum bandwidth is now deterministic and no longer sensitive to the access pattern.*

The maximum throughput of the SSD largely depends on the access pattern, mainly due to the asymmetric latency of different flash operations. For example, since flash reads typically have shorter latency than flash writes, the maximum read bandwidth is usually higher than the maximum write bandwidth (e.g., 3.5GB/s vs. 2.7GB/s in Samsung 970 Pro, see Table I). However, we find that the maximum bandwidth of ESSDs is deterministic and no longer sensitive to the access pattern. Figure 5 shows the throughput of two ESSDs and

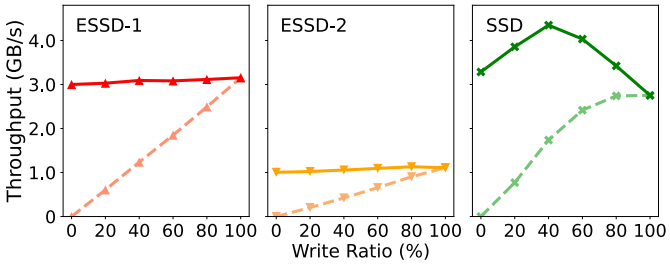


Fig. 5. **Throughput of two ESSDs and SSD under mixed read/write workloads with different write ratios.** The solid and dashed lines represent the total (i.e., read and write) throughput and the write throughput, respectively.

SSD under mixed read/write workloads. The workloads have different write ratios from 0 to 100, where 0 and 100 denote the random read and write workloads, respectively.

We can observe that, regardless of the specific write ratio, the throughput of two ESSDs is deterministically surrounded at the value guaranteed by the provider, i.e., 3.0GB/s and 1.1GB/s, respectively. In contrast, the throughput of the SSD is non-deterministic, varying between 2.5GB/s and 4.3GB/s in different write ratios. Note that this observation holds only for throughput and not for IOPS, as we find that the guaranteed IOPS in ESSDs is non-deterministic and is closely related to the I/O size.

The above results and analysis actually serve as a reminder to cloud storage users that, in contrast to the intricate performance engineering on the SSD, the throughput optimization target of the ESSD is straightforward and clear. For cloud storage users, this can deliver two implications.

First, since the throughput budget directly affects the system cost (see §II-C), to achieve higher cost-effectiveness through a smaller throughput budget, cloud software should *smooth the read/write I/Os to be evenly distributed across the timeline and below the guaranteed throughput budget* (**Implication #4**). This implication would be much more meaningful for software with frequent I/O bursts (i.e., I/O loads are very uneven across the timeline).

Second, to achieve higher cost-effectiveness, cloud software should also *re-evaluate I/O reduction techniques (e.g., compression [30], deduplication [31]) that were previously considered to impair performance* (**Implication #5**). The rationale behind this is that, for local SSDs with low latency, these I/O reduction techniques usually result in performance degradation due to high computational overhead [30], [31]. However, for ESSDs with network latency and software processing overhead that accompany cloud storage, the overhead of these techniques may not be the bottleneck, leading to not only cost reduction (i.e., accommodate workloads with a smaller throughput budget) but also performance improvement.

IV. Related Work

Depicting the performance characteristics of emerging storage devices is always a timeless topic. Prior works have investigated the performance features of HDD [32], flash-based SSD [27], Optane-based SSD [14], and flash-based SSD with new interfaces such as Key-Value (KV) [15] and Zoned Namespace (ZNS) [16]. Inspired by them, we first characterize the performance of the emerging cloud-based ESSD.

Some works introduce EBS's designs and evolution in storage, networking, and hardware acceleration [1], [3]–[5]. Li et al. characterize the I/O features of workloads on EBS from Alibaba Cloud and Tencent Cloud [2]. Cosine [33] searches for the best key-value storage engine with its cost models considering an input workload, a cloud budget, a target performance, and required cloud SLAs. Many works focus on data management between EBS and other faster/slower tiers (e.g., local SSD, object store such as Amazon S3 [34]). For example, Mutant [35] proposes to organize different SSTables in the key-value store into different storage tiers based on access frequencies. ROCKSMASH [36] proposes new cache and metadata structure designs to use local SSDs to store frequently accessed data and metadata while using EBS to hold the rest. MirrorKV [37] focuses on hybrid cloud storage consisting of EBS and object store, and proposes a series of designs to address the efficiency challenges of compaction and querying in this architecture.

Zhou et al. reveal the high latency variances in ESSDs when exceeding the paid IOPS budget and propose a range of I/O-regulated techniques in the key-value store to improve tail latency [38]. In contrast, we focus on the comprehensive performance characterization of ESSDs and deliver insights from many aspects (e.g., latency performance, GC, access pattern, throughput budget).

V. Conclusion and Future Work

This paper pioneers a thorough performance characterization of the cloud-based ESSD, and draws an unwritten contract that incorporates four counter-intuitive observations and five intriguing implications. We hope the unwritten contract could guide cloud storage users in revisiting their deployed cloud software designs, particularly, harnessing the distinct performance characteristics of ESSDs for better system performance.

Meanwhile, given the increasingly rapid growth of cloud computing and cloud storage, we believe this research direction deserves more investigation. We outline the future work as follows.

- **Comprehensiveness and generalizability.** We acknowledge that our current observations may not cover all the unique properties of ESSDs. In the future, we will continue to investigate, and also evaluate ESSDs from more cloud storage providers and validate our observations in this paper on these ESSDs.
- **Cloud-software-level exploration.** In the future, we will explore how cloud software can take advantage of the implications offered in this paper. In particular, we will first use RocksDB [39] (a popular persistent key-value store) as a case study. Although some works have explored building key-value stores on cloud storage (see §IV), these works may not fully exploit the potential of ESSDs.

REFERENCES

[1] R. Miao, L. Zhu, S. Ma, K. Qian, S. Zhuang, B. Li, S. Cheng, J. Gao, Y. Zhuang, P. Zhang *et al.*, "From luna to solar: the evolutions of the compute-to-storage networks in alibaba cloud," in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 753–766.

[2] J. Li, Q. Wang, P. P. Lee, and C. Shi, "An in-depth comparative analysis of cloud block storage workloads: Findings and implications," *ACM Transactions on Storage*, vol. 19, no. 2, pp. 1–32, 2023.

[3] W. Zhang, E. Xu, Q. Wang, X. Zhang, Y. Gu, Z. Lu, T. Ouyang, G. Dai, W. Peng, Z. Xu *et al.*, "What's the story in {EBS} glory: Evolutions and lessons in building cloud block store," in *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, 2024, pp. 277–291.

[4] Z. Wang, Y. Song, E. Xu, H. Wu, G. Tong, S. Sun, H. Li, J. Liu, L. Ding, R. Liu *et al.*, "Ransom access memories: Achieving practical ransomware protection in cloud with {DeftPunk}," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 687–702.

[5] J. Shu, K. Qian, E. Zhai, X. Liu, and X. Jin, "Burstable cloud block storage with data processing units," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 783–799.

[6] "Amazon AWS ESSD," https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volumes.html, 2024.

[7] "Alibaba Cloud ESSD," https://www.alibabacloud.com/help/en/ecs/user-guide/essds, 2024.

[8] "IDC Data Age 2025," https://www.seagate.com/files/www-content/our-story/trends/files/Seagate-WP-DataAge2025-March-2017.pdf, 2017.

[9] Z. Jiao, J. Bhimani, and B. S. Kim, "Wear leveling in ssds considered harmful," in *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, 2022, pp. 72–78.

[10] K. Zhao, W. Zhao, H. Sun, X. Zhang, N. Zheng, and T. Zhang, "{LDPC-in-SSD}: Making advanced error correction codes work effectively in solid state drives," in *11th USENIX Conference on File and Storage Technologies (FAST 13)*, 2013, pp. 243–256.

[11] J.-N. Yen, Y.-C. Hsieh, C.-Y. Chen, T.-Y. Chen, C.-L. Yang, H.-Y. Cheng, and Y. Luo, "Efficient bad block management with cluster similarity," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 503–513.

[12] "Amazon EBS volume types," https://aws.amazon.com/ebs/volume-types/, 2024.

[13] "Samsung 970 Pro," https://semiconductor.samsung.com/consumer-storage/internal-ssd/970pro/, 2024.

[14] K. Wu, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, "Towards an unwritten contract of intel optane {SSD}," in *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*, 2019.

[15] M. P. Saha, A. Maruf, B. S. Kim, and J. Bhimani, "Kv-ssd: What is it good for?" in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1105–1110.

[16] K. Doekemeijer, N. Tehrany, B. Chandrasekaran, M. Bjørling, and A. Trivedi, "Performance characterization of nvme flash devices with zoned namespaces (zns)," in *2023 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2023, pp. 118–131.

[17] "FIO benchmark," https://fio.readthedocs.io/en/latest/, 2024.

[18] N. Li, M. Hao, H. Li, X. Lin, T. Emami, and H. S. Gunawi, "Fantastic ssd internals and how to learn and use them," in *Proceedings of the 15th ACM International Conference on Systems and Storage*, 2022, pp. 72–84.

[19] S. Yan, H. Li, M. Hao, M. H. Tong, S. Sundararaman, A. A. Chien, and H. S. Gunawi, "Tiny-tail flash: Near-perfect elimination of garbage collection tail latencies in nand ssds," *ACM Transactions on Storage (TOS)*, vol. 13, no. 3, pp. 1–26, 2017.

[20] N. Elyasi, C. Choi, A. Sivasubramaniam, J. Yang, and V. Balakrishnan, "Trimming the tail for deterministic read performance in ssds," in *2019 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2019, pp. 49–58.

[21] H. Li, M. L. Putra, R. Shi, X. Lin, G. R. Ganger, and H. S. Gunawi, "Ioda: A host/device co-design for strong predictability contract on modern flash storage," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 263–279.

[22] D. Skourtis, D. Achlioptas, N. Watkins, C. Maltzahn, and S. Brandt, "Flash on rails: Consistent flash performance through redundancy," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 463–474.

[23] J. Kim, K. Lim, Y. Jung, S. Lee, C. Min, and S. H. Noh, "Alleviating garbage collection interference through spatial separation in all flash arrays," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 799–812.

[24] T. Jiang, G. Zhang, Z. Huang, X. Ma, J. Wei, Z. Li, and W. Zheng, "{FusionRAID}: Achieving consistent low latency for commodity {SSD} arrays," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 355–370.

[25] F. Chen, D. A. Koufaty, and X. Zhang, "Understanding intrinsic characteristics and system implications of flash memory based solid state drives," *ACM SIGMETRICS Performance Evaluation Review*, vol. 37, no. 1, pp. 181–192, 2009.

[26] C. Min, K. Kim, H. Cho, S.-W. Lee, and Y. I. Eom, "Sfs: random write considered harmful in solid state drives." in *FAST*, vol. 12, 2012, pp. 1–16.

[27] J. He, S. Kannan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The unwritten contract of solid state drives," in *Proceedings of the twelfth European conference on computer systems*, 2017, pp. 127–144.

[28] C. Lee, D. Sim, J. Hwang, and S. Cho, "{F2FS}: A new file system for flash storage," in *13th USENIX Conference on File and Storage Technologies (FAST 15)*, 2015, pp. 273–286.

[29] O. Rodeh, J. Bacik, and C. Mason, "Btrfs: The linux b-tree filesystem," *ACM Transactions on Storage (TOS)*, vol. 9, no. 3, pp. 1–32, 2013.

[30] A. Zuck, S. Toledo, D. Sotnikov, and D. Harnik, "Compression and {SSDs}: Where and how?" in *2nd Workshop on Interactions of NVM/Flash with Operating Systems and Workloads (INFLOW 14)*, 2014.

[31] W. Xia, H. Jiang, D. Feng, F. Douglis, P. Shilane, Y. Hua, M. Fu, Y. Zhang, and Y. Zhou, "A comprehensive study of the past, present, and future of data deduplication," *Proceedings of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016.

[32] S. W. Schlosser and G. R. Ganger, "Mems-based storage devices and standard disk interfaces: A square peg in a round hole?" in *FAST*, 2004, pp. 87–100.

[33] S. Chatterjee, M. Jagadeesan, W. Qin, and S. Idreos, "Cosine: a cloud-cost optimized self-designing key-value storage engine," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 112–126, 2021.

[34] "Amazon S3," https://aws.amazon.com/s3/, 2024.

[35] H. Yoon, J. Yang, S. F. Kristjansson, S. E. Sigurdarson, Y. Vigfusson, and A. Gavrilovska, "Mutant: Balancing storage cost and latency in lsm-tree data stores," in *Proceedings of the ACM Symposium on Cloud Computing*, 2018, pp. 162–173.

[36] P. Xu, N. Zhao, J. Wan, W. Liu, S. Chen, Y. Zhou, H. Albahar, H. Liu, L. Tang, and Z. Tan, "Building a fast and efficient lsm-tree store by integrating local storage with cloud storage," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 19, no. 3, pp. 1–26, 2022.

[37] Z. Wang and Z. Shao, "Mirrorkv: An efficient key-value store on hybrid cloud storage with balanced performance of compaction and querying," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, pp. 1–27, 2023.

[38] Y. Zhou, J. Zhou, S. Chen, P. Xu, P. Wu, Y. Wang, X. Liu, L. Zhan, and J. Wan, "Calcspar: A {Contract-Aware}{LSM} store for cloud storage with low latency spikes," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 451–465.

[39] "RocksDB: A persistent key-value store for fast storage environments," http://rocksdb.org/, 2024.